In [ ]:
```python
import random

import marimo as mo
from pysmt.shortcuts import (
    And,
    Bool,
    Equals,
    GE,
    Iff,
    Int,
    Ite,
    LT,
    Not,
    Or,
    Plus,
    Solver,
    Symbol,
)
from pysmt.typing import BOOL, INT
```

Resumo do Exercício 1: Game of Life como Máquina de Estados Finita

Contexto & Visão Geral

O Conway's Game of Life é um autómato celular. Aqui modifica-se para ter um **espaço de estados finito**: uma grelha $N \times N$ ($N > 16$) de células normais, rodeada por uma fronteira de células com valores constantes. O objetivo é construir uma **FSM** que represente o autómato e **verificar propriedades** (ou obter contra-exemplos).

Definição do autómato (do enunciado)

- **Células normais:** $(i, j) \in \{1..N\}^2$, booleanas (morta=0, viva=1).
- **Estado inicial:** todas mortas, exceto as de $i, j \leq 3$ (bloco 3×3 vivo).
- **Estado de erro:** todas as células normais mortas.
- **Células iniciais** ($2N + 1$): as que têm $i = 0$ ou $j = 0$ (com $i, j \in \{0..N\}$). Na verificação universal (por defeito), são **variáveis simbólicas livres** no Z3; em modo concreto, são geradas aleatoriamente com probabilidade $1/2$ de estarem vivas e mantêm-se constantes.
- **Células finais** ($2N + 3$): as que não são iniciais e têm um índice $= N + 1$. Constantemente **mortas**.
- **Regra B3/S23:** nasce se tem exatamente 3 vizinhos vivos; sobrevive se tem 2 ou 3; caso contrário morre/continua morta.

- **Vizinhança:** $\max(|i_0 - i_1|, |j_0 - j_1|) = 1$ (8 vizinhos).

Objetivos Principais

1. **Construir a FSM** (estado, estado inicial, relação de transição, estado de erro) com **duas interpretações** escolhidas por um parâmetro `modo`:
   - `"async"` (estrito): em cada transição **uma e só uma** célula normal transita efetivamente de estado ($\text{prox}[c] \neq \text{curr}[c]$);
   - `"sync"`: em cada transição **todas** as células normais são atualizadas simultaneamente.
2. **Provar (ou obter contra-exemplos)**:
   - (i) nunca se alcança o estado de erro;
   - (ii) nenhuma célula normal está permanentemente morta ou permanentemente viva (leitura forte: constantes no ciclo limite / loop da FSM).

Decisões de interpretação

- **Fronteira Simbólica por defeito (`FRONTEIRA_SIMBOLICA = True`):** As $2N + 1$ células iniciais são representadas por variáveis simbólicas do Z3, permitindo considerar todas as 2^{2N+1} configurações possíveis da fronteira. O solver procura uma atribuição da fronteira e uma execução que constituam um contraexemplo à propriedade. Se encontrar uma solução SAT, a propriedade universal é refutada.
- `async` **estrito (sem *stuttering* artificial):** A relação de transição força a célula escolhida a mudar efetivamente de valor. Isto cumpre rigorosamente o enunciado (exatamente uma célula muda de estado por transição).
- **Variável** `sel`: A variável `sel` em `declare` é uma variável auxiliar não-determinística de escolha do índice da célula para a transição assíncrona e não faz parte da configuração física da grelha.
- **Leitura forte da Propriedade (ii):** Uma célula está permanentemente morta (ou viva) se existir um traço que termine num loop (ciclo da FSM) onde a célula se mantém constantemente morta (ou viva) em todos os estados do loop ($G\ \text{morta}(c)$ ou $G\ \text{viva}(c)$ no ciclo). No modo assíncrono estrito, se o sistema atingir um ponto fixo (como o estado de erro onde todas as células estão mortas), este comporta-se como um estado terminal sem transições de saída ativas.
- **Ausência de Justiça (*Fairness*) no modo assíncrono:** No modo assíncrono, a existência de um ciclo em que uma célula permanece constante representa uma execução possível da FSM. Não é imposta uma condição de justiça (*fairness*) que obrigue todas as células a serem selecionadas ao longo do tempo.

Método

Bounded model checking com pySMT (solver Z3): `bmc_always` para (i) e `bmc_loop_constante` para (ii) na leitura forte. Desdobra-se a transição até K transições ($K+1$ estados) e pergunta-se ao solver se existe uma execução que viole o invariante ou que forme um loop com comportamento constante da célula.

Validação e Testes Automáticos

1. Contagem das células iniciais ($2N+1$) e finais ($2N+3$).
2. Regra B3/S23 em padrões conhecidos (blinker, bloco), verificada no próprio modelo.
3. Trajetória no modo `sync` validada face à simulação em Python (com fronteira concreta).
4. No modo `async` estrito, verificação de transição de exatamente uma célula por passo.

Alínea a): A FSM

In [ ]:
```python
# parâmetros
N = 17
SEED = 1
FRONTEIRA_SIMBOLICA = True  # True: verificação universal (Z3); False: fronteira aleatória com SEED
MODO = "sync"  # "sync" ou "async"

def vizinhos(i, j):
    # max(|i0-i1|, |j0-j1|) = 1: os 8 vizinhos
    return [
        (i + a, j + b)
        for a in (-1, 0, 1)
        for b in (-1, 0, 1)
        if (a, b) != (0, 0)
    ]

def gerar_fronteira_concreta(N, seed):
    # gera uma fronteira aleatória concreta para simulação/testes em Python
    rng = random.Random(seed)
    iniciais = {
        (i, j): rng.random() < 0.5
        for i in range(N + 1)
        for j in range(N + 1)
        if i == 0 or j == 0
    }
    finais = {
        (i, j): False
        for i in range(N + 2)
        for j in range(N + 2)
```

```
        if (i == N + 1 or j == N + 1) and (i, j) not in iniciais
    }
    return {**iniciais, **finais}, len(iniciais), len(finais)

celulas = [(i, j) for i in range(1, N + 1) for j in range(1, N + 1)]
iniciais_coords = [(i, j) for i in range(N + 1) for j in range(N + 1) if i == 0 or
j == 0]
finais_coords = [
    (i, j)
    for i in range(N + 2)
    for j in range(N + 2)
    if (i == N + 1 or j == N + 1) and (i, j) not in set(iniciais_coords)
]

n_iniciais = len(iniciais_coords)
n_finais = len(finais_coords)
borda_concreta, _, _ = gerar_fronteira_concreta(N, SEED)
```

Nesta parte definimos os parâmetros do problema, a vizinhança e a estrutura da fronteira da grelha.

Os parâmetros são `N` (o tamanho da grelha), `FRONTEIRA_SIMBOLICA` (que permite escolher entre uma fronteira simbólica e uma fronteira aleatória concreta), `SEED` (para a geração da fronteira aleatória) e `MODO` (`"sync"` ou `"async"`).

A função `vizinhos` devolve as 8 posições à volta da célula (i, j), correspondentes à condição $\max(|i_0 - i_1|, |j_0 - j_1|) = 1$, cobrindo a vizinhança de Moore na horizontal, vertical e diagonal.

A fronteira divide-se em:

- $2N + 1$ **células iniciais** ($i = 0 \vee j = 0$, com $i, j \in \{0, \ldots, N\}$): quando `FRONTEIRA_SIMBOLICA = True`, são representadas por símbolos do Z3 constantes ao longo do tempo, permitindo considerar todas as valorações possíveis. Caso contrário, são geradas aleatoriamente, com probabilidade $1/2$ de estarem vivas, utilizando a semente `SEED`.
- $2N + 3$ **células finais** (com pelo menos um índice igual a $N + 1$, não pertencendo às células iniciais): estão permanentemente mortas (`False`). A contagem resulta de $(N + 2)^2 - (N + 1)^2 = 2N + 3$.

In [ ]:
```
# símbolos globais para a fronteira inicial (constantes ao longo das transições)
borda_syms = {(i, j): Symbol(f"borda_{i}_{j}", BOOL) for (i, j) in
iniciais_coords}

def declare(i):
    # variáveis booleanas por célula normal + 'sel' (variável não-determinística
auxiliar de escolha da célula em async)
    state = {c: Symbol(f"s_{c[0]}_{c[1]}@{i}", BOOL) for c in celulas}
    state["sel"] = Symbol(f"sel@{i}", INT)
    return state
```

```
def viva(state, p):
    # célula normal: variável do estado; fronteira final: morta; fronteira
inicial: simbólica ou concreta
    if p in state:
        return state[p]
    if p in set(finais_coords):
        return Bool(False)
    return borda_syms[p] if FRONTEIRA_SIMBOLICA else Bool(borda_concreta[p])

def regra(state, c):
    n = Plus([Ite(viva(state, p), Int(1), Int(0)) for p in vizinhos(*c)])
    return Or(Equals(n, Int(3)), And(viva(state, c), Equals(n, Int(2))))

def init(state):
    return And([Iff(state[c], Bool(c[0] <= 3 and c[1] <= 3)) for c in celulas])

def trans_sync(curr, prox):
    # todas as células normais são atualizadas em simultâneo
    return And([Iff(prox[c], regra(curr, c)) for c in celulas])

def trans_async(curr, prox):
    # async estrito: a célula escolhida transita e MUDA efetivamente de estado
    # cumpre rigorosamente o enunciado: em cada transição uma e só uma célula
transita de estado
    escolha_valida = And(GE(curr["sel"], Int(0)), LT(curr["sel"],
Int(len(celulas))))
    return And(
        [escolha_valida]
        + [
            Ite(
                Equals(curr["sel"], Int(k)),
                And(Not(Iff(regra(curr, c), curr[c])), Iff(prox[c], regra(curr,
c))),
                Iff(prox[c], curr[c]),
            )
            for k, c in enumerate(celulas)
        ]
    )

# o parâmetro MODO escolhe a relação de transição a usar por defeito
trans = trans_sync if MODO == "sync" else trans_async

def erro(state):
    # estado de erro: todas as células normais mortas
    return Not(Or([state[c] for c in celulas]))
```

Nesta parte construímos a FSM como um sistema de transição, com as funções `declare`, `init` e `trans`, bem como o predicado `erro`.

A função `declare(i)` cria a i-ésima cópia das variáveis de estado: uma variável booleana por cada célula normal e uma variável inteira `sel`, que é utilizada como mecanismo auxiliar de escolha no modo assíncrono.

A função `viva` permite aceder ao valor de uma célula: se for normal, devolve a respetiva variável de estado; se pertencer à fronteira, devolve o valor correspondente, que pode ser simbólico ou constante, dependendo da configuração.

A função `regra` conta os vizinhos vivos de uma célula e aplica a regra B3/S23: a célula fica viva se tiver exatamente 3 vizinhos vivos, ou se já estiver viva e tiver exatamente 2.

O predicado `init` fixa o bloco 3×3 vivo e todas as restantes células normais mortas. O predicado `erro` verifica se todas as células normais estão mortas.

Existem duas relações de transição:

- `trans_sync` : todas as células normais são atualizadas simultaneamente, de acordo com a regra B3/S23.
- `trans_async` : a variável `sel` indica qual a célula escolhida para atualização. Essa célula tem obrigatoriamente de mudar de valor, passando a assumir o resultado da regra, enquanto todas as outras permanecem inalteradas. Como `sel` é uma variável não determinística, é o solver que escolhe a célula a atualizar em cada transição.

A relação de transição `trans` utilizada nas verificações é escolhida através do parâmetro `MODO` , podendo corresponder ao modo síncrono ou assíncrono.

Testes da FSM

In [ ]:
```
def sat(formula):
    with Solver(name="z3") as solver:
        solver.add_assertion(formula)
        return solver.solve()

def teste_fronteira():
    for n in (4, 17, 20):
        b, ini, fin = gerar_fronteira_concreta(n, 0)
        assert ini == 2 * n + 1 and fin == 2 * n + 3
        assert len(b) == ini + fin
        assert all(not v for p, v in b.items() if n + 1 in p)
    assert n_iniciais == 2 * N + 1 and n_finais == 2 * N + 3
    assert len(vizinhos(5, 5)) == 8 and (5, 5) not in vizinhos(5, 5)
    print("fronteira: 2N+1 células iniciais, 2N+3 finais e 8 vizinhos")

def verificar_padrao(vivas, seguintes):
    s = declare(0)
    janela = [(i, j) for i in range(7, 12) for j in range(7, 12)]
    estado = And([Iff(s[c], Bool(c in vivas)) for c in celulas])
    errada = Or([Not(Iff(regra(s, c), Bool(c in seguintes))) for c in janela])
    assert not sat(And(estado, errada))

def teste_regra():
```

```
    horizontal = {(9, 8), (9, 9), (9, 10)}
    vertical = {(8, 9), (9, 9), (10, 9)}
    bloco = {(9, 9), (9, 10), (10, 9), (10, 10)}
    verificar_padrao(horizontal, vertical)
    verificar_padrao(vertical, horizontal)
    verificar_padrao(bloco, bloco)
    print("regra B3/S23: blinker e bloco")

teste_fronteira()
teste_regra()
```

Nesta parte testamos a fronteira e a regra B3/S23.

Em `teste_fronteira` confirmamos, para vários valores de `N`, que existem $2N + 1$ células iniciais e $2N + 3$ células finais. Verificamos também que os dois conjuntos não se sobrepõem, que as células finais estão mortas e que a função `vizinhos` devolve 8 posições para uma célula interior.

Em `teste_regra` verificamos a regra B3/S23 em dois padrões conhecidos, colocados no meio da grelha, longe da fronteira. Um **blinker** (três células em linha) tem de alternar entre a posição horizontal e a vertical, enquanto um **bloco** (2×2) tem de permanecer inalterado.

A função `verificar_padrao` fixa o estado com apenas as células indicadas como vivas e pergunta ao solver se existe alguma célula da janela analisada em que o resultado da regra seja diferente do esperado. Se a fórmula for insatisfazível (UNSAT), significa que não foi encontrado qualquer resultado diferente do esperado nesse padrão.

A função `sat` verifica simplesmente se uma fórmula é satisfazível, seguindo a abordagem utilizada nas fichas.

In [ ]:
```
normais = set(celulas)

def passo_py(vivos):
    novos = set()
    for i, j in celulas:
        n = 0
        for p in vizinhos(i, j):
            n += (p in vivos) if p in normais else borda_concreta[p]
        if n == 3 or ((i, j) in vivos and n == 2):
            novos.add((i, j))
    return frozenset(novos)

def teste_sync_vs_python(K=5):
    S = [declare(i) for i in range(K + 1)]
    with Solver(name="z3") as solver:
        solver.add_assertion(init(S[0]))
        # asserção dos valores concretos da borda para testar com a simulação
Python
        for p, v in borda_concreta.items():
            if p in borda_syms:
                solver.add_assertion(Iff(borda_syms[p], Bool(v)))
        for i in range(K):
```

```
            solver.add_assertion(trans_sync(S[i], S[i + 1]))
        assert solver.solve()
        esperado = frozenset(c for c in celulas if c[0] <= 3 and c[1] <= 3)
        for i in range(K + 1):
            obtido = frozenset(c for c in celulas if solver.get_value(S[i]
[c]).is_true())
            assert obtido == esperado, f"difere no passo {i}"
            esperado = passo_py(esperado)
    print(f"sync: o modelo coincide com a simulação em Python em {K} passos")

def teste_async_uma_celula():
    s0, s1 = declare(0), declare(1)
    mudou = Plus([Ite(Not(Iff(s0[c], s1[c])), Int(1), Int(0)) for c in celulas])
    assert not sat(And(trans_async(s0, s1), GE(mudou, Int(2))))  # nunca mudam
duas
    assert sat(And(init(s0), trans_async(s0, s1), Equals(mudou, Int(1))))  # muda
exatamente uma
    print("async estrito: exatamente uma célula muda por transição ativamente")

teste_sync_vs_python()
teste_async_uma_celula()
```

Nesta parte testamos as duas relações de transição.

Em `teste_sync_vs_python` pedimos ao solver os estados do modelo `sync`, a partir do estado inicial, durante 5 transições, e comparamo-los com `passo_py`, uma simulação em Python do mesmo processo. Para garantir uma comparação nas mesmas condições, fixamos os valores da fronteira aos valores concretos gerados com a semente `SEED`. Como a atualização em `sync` é determinística, a configuração das células normais tem de coincidir em todos os passos.

Em `teste_async_uma_celula` verificamos duas propriedades de `trans_async`: é impossível que duas ou mais células mudem de valor na mesma transição (a fórmula é insatisfazível), e existe pelo menos uma transição válida a partir do estado inicial em que exatamente uma célula muda (a fórmula é satisfazível). Assim confirmamos que a relação de transição respeita a atualização assíncrona estrita.

Estes testes utilizam diretamente `trans_sync` e `trans_async`, pelo que são independentes do valor de `MODO`.

Alínea b): As propriedades

In [ ]:
```
def estado_vivas(model, st):
    return frozenset(c for c in celulas if model.get_py_value(st[c]))

def mostrar_estado(vivas):
    for i in range(1, N + 1):
        print(" ".join("#" if (i, j) in vivas else "." for j in range(1, N + 1)))
```

```
def bmc_always(declare, init, trans, inv, K):
    # Aloca K + 1 estados para verificar K transições (do estado 0 ao estado K)
    states = [declare(i) for i in range(K + 1)]
    with Solver(name="z3") as s:
        s.add_assertion(init(states[0]))

        # Verificação do estado inicial (k = 0)
        s.push()
        s.add_assertion(Not(inv(states[0])))
        if s.solve():
            model = s.get_model()
            return [estado_vivas(model, states[0])]
        s.pop()

        # Verificação dos estados k = 1 até K
        for k in range(1, K + 1):
            s.add_assertion(trans(states[k - 1], states[k]))
            s.push()
            s.add_assertion(Not(inv(states[k])))
            if s.solve():
                model = s.get_model()
                return [estado_vivas(model, st) for st in states[: k + 1]]
            s.pop()
    return None

def bmc_loop_constante(declare, init, trans, c, valor_constante, K):
    # Procura prefixos com até K transições (K + 1 estados),
    # seguidos de uma transição de fecho do ciclo (0 <= j <= k).
    states = [declare(i) for i in range(K + 1)]
    with Solver(name="z3") as s:
        s.add_assertion(init(states[0]))
        for k in range(1, K + 1):
            s.add_assertion(trans(states[k - 1], states[k]))

            # states[k] regressa a um states[j] (0 <= j <= k)
            # E em todos os estados m do loop (j <= m <= k), a célula c tem o
valor_constante.
            loop_constante = Or([
                And(
                    trans(states[k], states[j]),
                    And([Iff(states[m][c], Bool(valor_constante)) for m in
range(j, k + 1)]),
                    )
                    for j in range(k + 1)
            ])

            s.push()
            s.add_assertion(loop_constante)
            if s.solve():
                model = s.get_model()
                traco = [estado_vivas(model, st) for st in states[: k + 1]]
                inicio = next(
                    j
                    for j in range(k + 1)
                    if model.get_py_value(trans(states[k], states[j]))
```

```
                    and all(
                        model.get_py_value(Iff(states[m][c],
Bool(valor_constante)))
                        for m in range(j, k + 1)
                    )
                )
                return traco, inicio
            s.pop()
    return None
```

Nesta parte definimos as rotinas de Bounded Model Checking (BMC), adaptadas à FSM construída:

- `bmc_always` : verifica a propriedade de invariância $G\,\mathrm{inv}$, procurando uma execução que alcance um estado onde o invariante é falso. Verifica sucessivamente execuções até ao limite de K transições ($K+1$ estados).
- `bmc_loop_constante` : procura um traço iniciado no estado inicial, com um prefixo de até K transições, que possa ser fechado através de uma transição para um estado anterior ou para o próprio estado final. Desta forma, são identificados ciclos de diferentes comprimentos, incluindo ciclos de comprimento 1, nos quais a célula c mantém constantemente o valor indicado em todos os estados do ciclo.

Testes

In [ ]:
```
def nunca_erro(state):
    return Not(erro(state))

def verificar_i(trans, K):
    contra = bmc_always(declare, init, trans, nunca_erro, K)
    if contra is None:
        print(f"(i) NENHUM ERRO ENCONTRADO ate {K} transicoes (pesquisa limitada a
K={K})")
    else:
        print(f"(i) FALSA: o estado de erro e alcancado ao fim de {len(contra) -
1} transicoes")
        print("estado inicial:")
        mostrar_estado(contra[0])
        print("estado de erro:")
        mostrar_estado(contra[-1])

def verificar_ii(trans, c, K):
    print(f"(ii) célula {c} (leitura forte no loop):")

    res_morta = bmc_loop_constante(declare, init, trans, c, False, K)
    if res_morta is not None:
        traco, inicio = res_morta
        ciclo_n = len(traco) - inicio
        print(
            f"  FICA PERMANENTEMENTE MORTA: ciclo encontrado com {ciclo_n} estados
"
            f"(traço total com {len(traco)} estados), a começar no estado
```

```
{inicio}."
        )
    else:
        print(f"  não foi encontrado um ciclo com a célula permanentemente morta
até ao limite K={K}")

    res_viva = bmc_loop_constante(declare, init, trans, c, True, K)
    if res_viva is not None:
        traco, inicio = res_viva
        ciclo_n = len(traco) - inicio
        print(
            f"  FICA PERMANENTEMENTE VIVA: ciclo encontrado com {ciclo_n} estados
"
            f"(traço total com {len(traco)} estados), a começar no estado
{inicio}."
        )
    else:
        print(f"  não foi encontrado um ciclo com a célula permanentemente viva
até ao limite K={K}")
```

As funções `verificar_i` e `verificar_ii` utilizam estas rotinas para verificar as propriedades do enunciado:

- `verificar_i` verifica se o estado de erro nunca é alcançado, considerando o invariante $\neg erro$.
- `verificar_ii` procura, para uma célula escolhida, um ciclo em que permaneça sempre morta e outro em que permaneça sempre viva.

As duas rotinas de BMC utilizam um único solver por chamada, recorrendo a `push` e `pop` para testar sucessivamente os diferentes limites k, sem ter de reconstruir o problema de cada vez.

É importante notar que, por se tratar de Bounded Model Checking, não encontrar um contraexemplo até ao limite K não prova, por si só, que a propriedade seja verdadeira.

In [ ]:
```
K_limite = 30

print(f"=== Verificação no modo selecionado: MODO = '{MODO}' ===")

# Verificação do modo escolhido pelo parâmetro
verificar_i(trans, K_limite)
verificar_ii(trans, (N, N), K_limite)

if MODO == "async":
    celula_extra = (2, 2)
else:
    celula_extra = (4, 4)
verificar_ii(trans, celula_extra, K_limite)

# Verificação do outro modo, para comparação
if MODO == "sync":
    outro_trans = trans_async
    outro_modo = "async (estrito)"
```

```
else:
    outro_trans = trans_sync
    outro_modo = "sync"

print(f"\n--- Comparação com MODO '{outro_modo}' ---")

verificar_i(outro_trans, K_limite)
verificar_ii(outro_trans, (N, N), K_limite)
```

Análise dos resultados obtidos nos dois modos:

`sync` (com fronteira simbólica por defeito).

- **(i) Estado de erro:** A verificação considera todas as configurações possíveis da fronteira inicial. Se o solver encontrar uma execução que alcance o estado de erro, a propriedade é falsa, pois existe pelo menos uma configuração da fronteira que permite extinguir todas as células normais.
- **(ii) Permanência das células:** Na leitura forte baseada em ciclos, procura-se identificar células que mantenham constantemente o mesmo valor ao longo de um ciclo da FSM. Um ciclo encontrado constitui um contraexemplo à afirmação de que nenhuma célula permanece permanentemente morta ou viva.

`async` (estrito).

- **(i) Estado de erro:** O solver procura uma sequência de atualizações assíncronas em que exatamente uma célula muda em cada transição, até eventualmente atingir o estado de erro. Uma execução encontrada pelo solver constitui um contraexemplo válido à propriedade.
- **(ii) Permanência das células:** Procura-se um ciclo genuíno da FSM no qual determinada célula mantenha sempre o mesmo valor. Como as transições são estritas, os pontos fixos são estados terminais sem transições de saída, não sendo considerados ciclos artificiais de *stuttering*.

Conclusão. Os resultados das duas propriedades são analisados separadamente para os modos síncrono e assíncrono. Quando o solver encontra um contraexemplo, a propriedade correspondente é refutada. A ausência de contraexemplos até ao limite K representa apenas uma pesquisa limitada, não uma prova de validade da propriedade.

Registo de Utilizacao de LLM

- **Claude:** usado para estruturar a resolucao por fases e resolver erros simples, maioritariamente de sintaxe durante a resolução do exercicio.
- **Gemini:** usado para, apos resolver cada parte do exercicio, corrigir, dar dicas de como resolver alguns problemas e erros dando a sua resolução para ser adaptada.
- **ChatGPT:** usado para comparar as minhas explicaçoes com explicaçoes criadas pelo gemini e escolher ou gerar a mais adequada para cada situação.

Links:

- **Gemini:** https://share.gemini.google/YCvyICcK8so3
- **ChatGPT:** https://chatgpt.com/share/6aea21b4-9da4-83ed-a5f8-039f2d0431c3